

CSC435 Programming Assignment - II

Chameleon resource used: compute_cascadelake_r

1. Datastructure used:

a. Map.

- i. There are two structures in the map data structure. I have used both of them.
- ii. First I have used `unordered_map` to store the word and it's associating data.
- iii. Second I have used `map` to store the frequency count in (descending order) and the filepath associate with the word.

```
(struct IndexingData
{
    // Use map sorted by frequency in descending order
    std::map<unsigned int, std::string, std::greater<unsigned int>>
    list;

    ...
};)
```

- iv. I have used these data structures for two reasons, `unordered_map` helps to retrieve the data in $O(1)$ (very efficient retrieval) and `map` helps with reordered storage of data and also faster retrieval.

2. Creation of global index(in safe way):

a. Mutex.

- i. I have used `std::mutex mtx;` in the `IndexStore` class.
- ii. When inserting global index/updating the global index, I look the class object using mutex which makes sure R/W operations are done by only that specific thread which has locked the class.

Continue to Next Page

CSC435 Programming Assignment - II

3. Indexing and search performance:

- a. How is the performance influenced?
 - i. As the dataset size increases the need of memory and computation increases, which is a direct factor affecting indexing and search(in some cases) performance.
 - ii. Check the plot diagrams in the end, how indexing and search is affected based on dataset size.
- b. Performance exhibition from the solution:
 - i. The performance of indexing and search were very much related with each other's run-time.
 - ii. As it took so much time for indexing, most of the operations were performed during this stage.
 - iii. As the dataset size was huge the above stage took so much time for loading, indexing, organizing and storing the data ready for the search operations (which needs to be as fast as possible, instantaneous in some cases).
 - iv. Searching was fast because the data is organized in way where the elements are retrieved very quickly.

4. Challenges and Conclusion:

- a. Things which could have done better.
 - i. Instead of using `unordered_map`, I could have used `concurrent_unordered_map`, which could have made me avoid using `std::mutex`, which could have ultimately increased the performance of the application.
 - ii. Regarding the data structures, if I would have stored data more efficiently, may be the performance of indexing could have been much better.
- b. Major Challenges faced.
 - i. One of major challenge which I have faced was correctly and in optimized way of storing the words.

CSC435 Programming Assignment - II

1. Initially I thought I'll replace separators and then remove delimiters based on the assignment conditions.
 2. After a long run or thought I released that it was better to ignore those characters and just store the words as soon as some invalid character is hit.
- ii. One of other major challenge was how to store the indexing data?
1. Do I store it in a vector of map? Do I store it in a map of vectors? Do I store it in an array? Do I store it in a Tree?
 2. If I'm sorting them in any container do I need to maintain their order?
 3. In case if the order is not sorted how do I perform the search? Do I just sort it before the search?

There were many more other questions/issues arose, but finally the solution was to use a map which holds key-pair values and also stores them in an orderly fashion on the insertion.

Continue to Next Page

CSC435 Programming Assignment - II

Screen snips of the plot data

[illegible]

Continue to Next Page

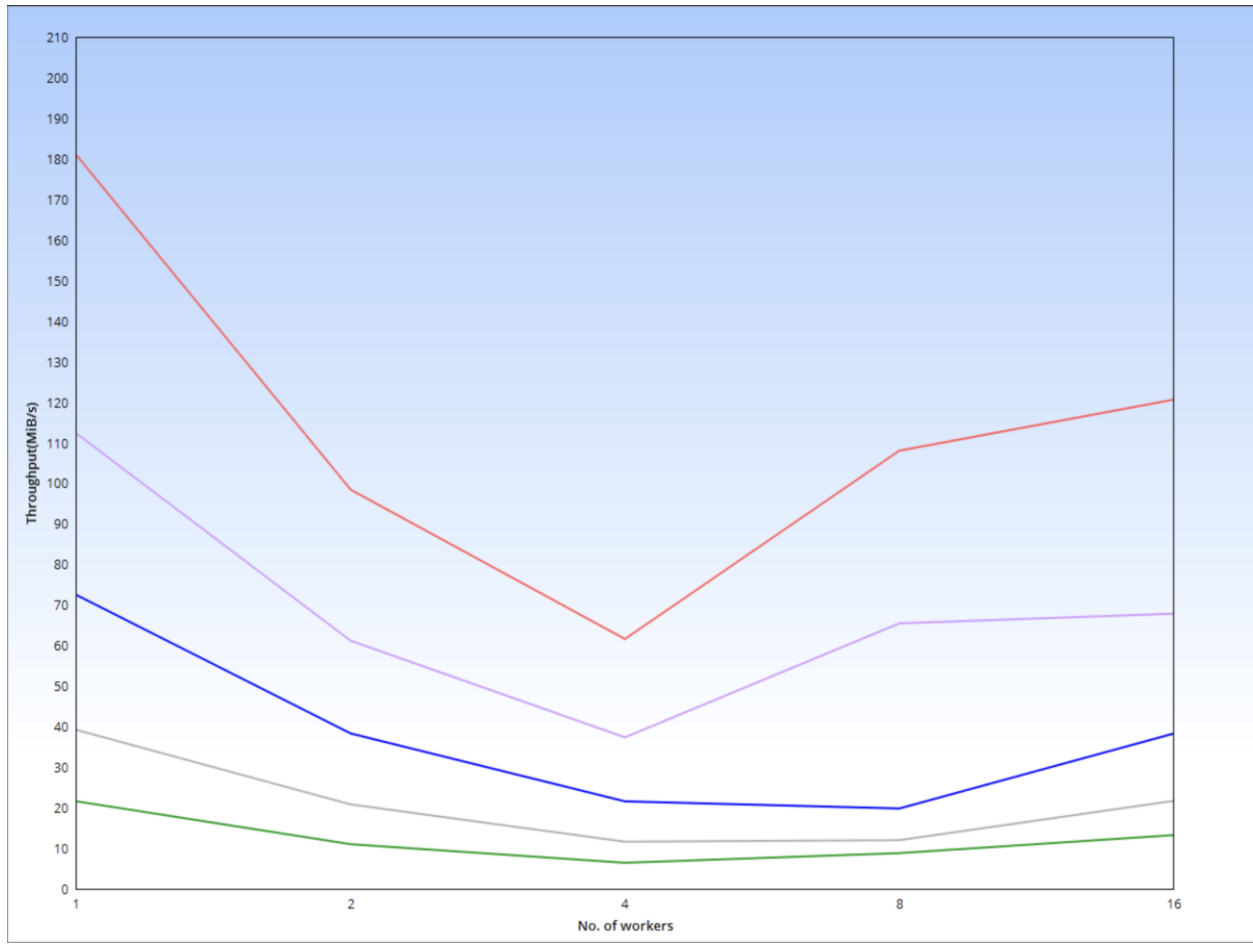
Student Name: Kedar Chandulur

DePaul Student ID: 2114590

Student Email: kchandul@depaul.edu

CSC435 Programming Assignment - II

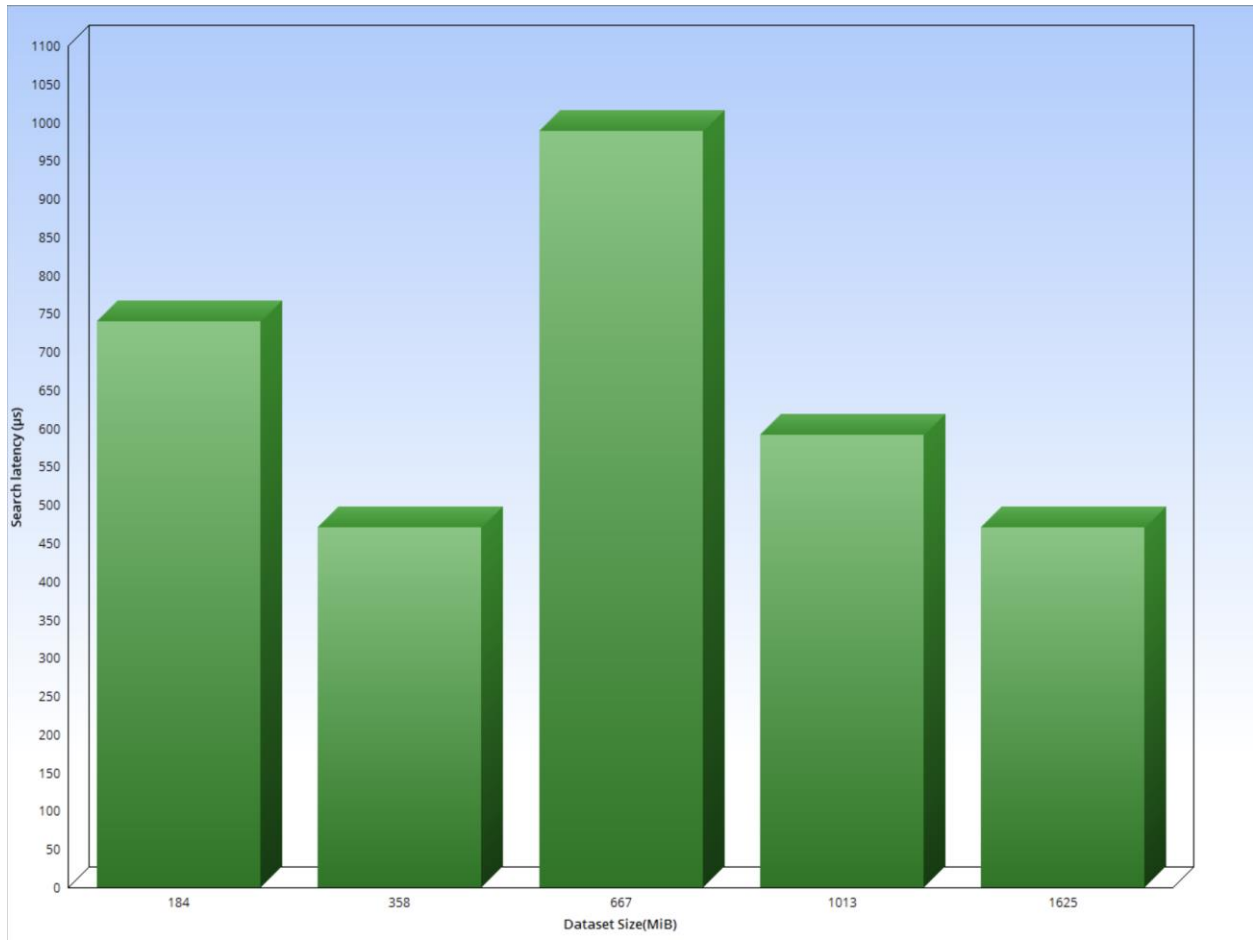
Screen snips of the insertion plot



Continue to Next Page

CSC435 Programming Assignment - II

Screen snips of the search plot



Student Name: Kedar Chandulur

DePaul Student ID: 2114590

Student Email: kchandul@depaul.edu